

```

namespace progiceTask3
{
    // Interface
    1 reference
    interface ISighting
    {
        4 references
        void DisplaySighting();
    }

    // Abstract base class
    4 references
    abstract class Animal : ISighting
    {
        4 references
        public string EnglishOrCommonName { get; set; }
        4 references
        public string LatinOrScientificName { get; set; }
        6 references
        public DateTime DateLastSeen { get; set; }
        4 references
        public abstract void DisplaySighting();
    }

    // Bird class
    3 references
    class Bird : Animal
    {
        3 references
        public override void DisplaySighting()
        {
            Console.WriteLine($"Bird - English Name: {EnglishOrCommonName}, Latin Name: {LatinOrScientificName}, Last Sighted: {DateLastSeen.ToShortDateString()}");
        }
    }

    // Butterfly class
    3 references
    class Butterfly : Animal
    {
        3 references
        public override void DisplaySighting()
        {
            Console.WriteLine($"Butterfly - Common Name: {EnglishOrCommonName}, Scientific Name: {LatinOrScientificName}, Last Seen: {DateLastSeen.ToShortDateString()}");
        }
    }
}

```

```

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        List<Animal> animals = new List<Animal>();

        bool running = true;

        while (running)
        {
            Console.WriteLine("\nChoose an option:");
            Console.WriteLine("1. Add Bird");
            Console.WriteLine("2. Add Butterfly");
            Console.WriteLine("3. Display All Sightings and Percentages");
            Console.WriteLine("4. Exit");
            Console.Write("Your choice: ");
            string choice = Console.ReadLine();

            switch (choice)
            {
                case "1":
                    Bird bird = new Bird();
                    Console.Write("English Name: ");
                    bird.EnglishOrCommonName = Console.ReadLine();

                    Console.Write("Latin Name: ");
                    bird.LatinOrScientificName = Console.ReadLine();

                    Console.Write("Date Last Sighted (yyyy-mm-dd): ");
                    if (DateTime.TryParse(Console.ReadLine(), out DateTime birdDate))
                        bird.DateLastSeen = birdDate;
                    else
                    {
                        Console.WriteLine("Invalid date. Using today's date.");
                        bird.DateLastSeen = DateTime.Today;
                    }

                    animals.Add(bird);
                    break;

                case "2":
                    Butterfly butterfly = new Butterfly();

```

```

        Console.WriteLine("Common Name: ");
        butterfly.EnglishOrCommonName = Console.ReadLine();

        Console.WriteLine("Scientific Name: ");
        butterfly.LatinOrScientificName = Console.ReadLine();

        Console.WriteLine("Date Last Seen (yyyy-mm-dd): ");
        if (DateTime.TryParse(Console.ReadLine(), out DateTime butterflyDate))
            butterfly.DateLastSeen = butterflyDate;
        else
        {
            Console.WriteLine("Invalid date. Using today's date.");
            butterfly.DateLastSeen = DateTime.Today;
        }

        animals.Add(butterfly);
        break;

case "3":
    Console.WriteLine("\n--- All Sightings ---");
    foreach (var a in animals)
        a.DisplaySighting();

    int birdCount = 0;
    int butterflyCount = 0;

    foreach (var a in animals)
    {
        if (a is Bird) birdCount++;
        if (a is Butterfly) butterflyCount++;
    }

    int total = birdCount + butterflyCount;

    try
    {
        if (total == 0)
            throw new DivideByZeroException("No entries found.");

        double birdPercentage = (double)birdCount / total * 100;
        double butterflyPercentage = (double)butterflyCount / total * 100;

        Console.WriteLine($"Percentage of Birds: {birdPercentage:F2}%");
        Console.WriteLine($"Percentage of Butterflies: {butterflyPercentage:F2}%");
    }

```

```

    }
    catch (DivideByZeroException ex)
    {
        Console.WriteLine($"Cannot calculate percentages: {ex.Message}");
    }
    break;

case "4":
    running = false;
    break;

default:
    Console.WriteLine("Invalid choice.");
    break;
}

Console.WriteLine("Goodbye!");
}
}
}

```

In the bird class I overrode the DisplaySighting method to show that it's a bird and to display the information about the bird. In the butterfly class I overrode the DisplaySighting method to show that it's a butterfly and to display the Common name, Scientific name, and the last sighting date.